

google / magika

Q

BYE10

<> Code Issues 97 Pull requests 5 Actions Projects Security Insights

👁

🔗

★

Detect file content types with deep learning

[google.github.io/magika/](#)

📄 Apache-2.0 license

📄 Code of conduct

📄 Security policy

★ 8.5k stars 🍴 444 forks 👁 38 watching 🧑 Branches 📉 Activity 📅 Custom properties 📁 Tags

🌐 Public repository

🧑 main 6 Branches 23 Tags 🧑 📁

🔍 Go to file t Go to file + Add file Code ...

🧑 reyammer Merge pull request #1060 from google/badges 055f10a · 14 hours ago 🔄

📁 .cargo	Update Rust CLI to model v2 (#555)	9 months ago
📁 .github	build(deps): bump ossf/scorecard-action fro...	4 days ago
📁 assets	assets: new standard_v3_3 model	20 hours ago
📁 docs	js: update paths names to follow best practices	5 days ago
📁 go	go: add a Go library for Magika	4 months ago
📁 js	js: update changelog	19 hours ago
📁 python	docs: add npm badge	14 hours ago
📁 rust	rust: update to standard_v3_3	19 hours ago
📁 tests_data	tests: add reference for standard_v3_3	19 hours ago
📁 website	website: trim text before processing	19 hours ago
📄 .dockerignore	improve instructions for docker	last year
📄 .gitattributes	gh: disable crlf conversion on all platforms via...	3 months ago
📄 .gitignore	update gitignore to allow .python-version	7 months ago
📄 CITATION.cff	bibtex: add dtao and other small fixes	7 months ago
📄 CONTRIBUTING.md	Update CONTRIBUTING.md	last year
📄 Dockerfile	improve instructions for docker	last year
📄 LICENSE	first external commit	last year
📄 README.md	docs: add npm badge	14 hours ago

Magika

🐍 pypi v0.6.1

📦 npm v0.3.2

📄 license Apache-2.0

🐍 python 3.8 | 3.9 | 3.10 | 3.11 | 3.12

🛡️ openssf best practices passing

🔍 CodeQL passing

🐍 Python - build package passing

📄 downloads/month 376k

📄 downloads 2M

Magika is a novel AI-powered file type detection tool that relies on the recent advance of deep learning to provide accurate detection. Under the hood, Magika employs a custom, highly optimized model that only weighs about a few MBs, and enables precise file identification within milliseconds, even when running on a single CPU. Magika has been trained and evaluated on a dataset of ~100M samples across 200+ content types (covering both binary and textual file formats), and it achieves an average ~99% accuracy on our test set.

Here is an example of what Magika command line output looks like:

```
$ magika -r examples/
examples/README.md: Markdown document (text)
examples/bmp.bmp: BMP image data (image)
examples/code.asm: Assembly (code)
examples/code.py: Python source (code)
examples/doc.docx: Microsoft Word 2007+ document (document)
examples/doc.ini: INI configuration file (text)
examples/elf64.elf: ELF executable (executable)
examples/flac.flac: FLAC audio bitstream data (audio)
examples/java.class: Java compiled bytecode (executable)
examples/jpg.jpg: JPEG image data (image)
examples/pdf.pdf: PDF document (document)
examples/pe32.exe: PE executable (executable)
examples/png.png: PNG image data (image)
examples/tar.tar: POSIX tar archive (archive)
examples/webm.webm: WebM data (video)
```

Magika is used at scale to help improve Google users' safety by routing Gmail, Drive, and Safe Browsing files to the proper security and content policy scanners, processing hundreds billions samples on a weekly basis. Magika has also been integrated with [VirusTotal \(example\)](#) and [abuse.ch \(example\)](#).

For more context you can read our initial [announcement post on Google's OSS blog](#), and you can read more in our [research paper](#), published at the IEEE/ACM International Conference on Software Engineering (ICSE) 2025.

You can try Magika without installing anything by using our [web demo](#), which runs locally in your browser! (Note: the website runs an older version of the model; but while results may differ, it is still useful to get an idea of Magika's capabilities.)

Important

- The documentation on GitHub refers to the latest, potentially unreleased and unstable version of Magika. The latest stable release of the `magika` Python package is `0.6.1`, and you can consult the associated documentation [here](#). You can install the latest stable version with: `pip install magika`.
- A detailed changelog and migration guidelines from older versions are discussed [here](#).
- Help testing the latest release candidate is very appreciated! See the available candidates [here](#) and the recent changes in the [CHANGELOG.md](#). You can install the latest release candidate with `pip install --pre magika`.

Highlights

- Available as a command line tool written in Rust, a Python API, and additional bindings for Rust, JavaScript/TypeScript (with an experimental npm package, which powers our [web demo](#)), and GoLang (WIP).
- Trained and evaluated on a dataset of ~100M files across [200+ content types](#).
- On our test set, Magika achieves ~99% average precision and recall, outperforming existing approaches.
- After the model is loaded (which is a one-off overhead), the inference time is about 5ms per file, even when run on a single CPU.
- You can invoke Magika with even thousands of files at the same time. You can also use `-r` for recursively scanning a directory.
- Near-constant inference time, independently from the file size; Magika only uses a limited subset of the file's content.
- Magika uses a per-content-type threshold system that determines whether to "trust" the prediction for the model, or whether to return a generic label, such as "Generic text document" or "Unknown binary data".
- The tolerance to errors can be controlled via different prediction modes, such as `high-confidence`, `medium-confidence`, and `best-guess`.
- The client and the bindings are already open source, and more is coming soon!

Table of Contents

1. [Getting Started](#)
 - i. [Installation](#)
 - ii. [Usage](#)
 - a. [Command line client](#)

b. [Python module and other bindings](#)

2. [Documentation](#)
3. [Bindings](#)
4. [Development Setup](#)
5. [Known Limitations & Contributing](#)
6. [Additional Resources](#)
7. [Research Paper and Citation](#)
8. [Security vulnerabilities](#)
9. [License](#)
10. [Disclaimer](#)

Getting Started

Installation

Magika is available as `magika` on PyPI:

```
$ pip install magika
```



If you intend to use Magika only as a command line, you may want to use `$ pipx install magika` instead.

If you want to test out the latest release candidate, you can install with `pip install --pre magika`.

If you want to test Magika within a Docker container, you can run:

```
git clone https://github.com/google/magika
cd magika/
docker build -t magika .
docker run -it --rm -v $(pwd):/magika magika -r /magika/tests_data/basic
```



Usage

Command line client

Magika is available as a command line tool, written in Rust (the `magika` python package ships this client). Here below are a few examples on how to use it, and the output of `magika --help`, which documents the list of options. See [here](#) for developer notes and documentation on how to install the Rust client via `cargo`.

Examples:

```
$ cd tests_data/basic && magika -r *
asm/code.asm: Assembly (code)
batch/simple.bat: DOS batch file (code)
c/code.c: C source (code)
css/code.css: CSS source (code)
csv/magika_test.csv: CSV document (code)
dockerfile/Dockerfile: Dockerfile (code)
docx/doc.docx: Microsoft Word 2007+ document (document)
epub/doc.epub: EPUB document (document)
epub/magika_test.epub: EPUB document (document)
flac/test.flac: FLAC audio bitstream data (audio)
handlebars/example.handlebars: Handlebars source (code)
html/doc.html: HTML document (code)
ini/doc.ini: INI configuration file (text)
javascript/code.js: JavaScript source (code)
jinja/example.j2: Jinja template (code)
jpeg/magika_test.jpg: JPEG image data (image)
json/doc.json: JSON document (code)
latex/sample.tex: LaTeX document (text)
makefile/simple.Makefile: Makefile source (code)
markdown/README.md: Markdown document (text)
[...]
```



```
$ magika ./tests_data/basic/python/code.py --json
[
```



```
{
  "path": "./tests_data/basic/python/code.py",
  "result": {
    "status": "ok",
    "value": {
      "dl": {
        "description": "Python source",
        "extensions": [
          "py",
          "pyi"
        ],
        "group": "code",
        "is_text": true,
        "label": "python",
        "mime_type": "text/x-python"
      },
      "output": {
        "description": "Python source",
        "extensions": [
          "py",
          "pyi"
        ],
        "group": "code",
        "is_text": true,
        "label": "python",
        "mime_type": "text/x-python"
      },
      "score": 0.753000020980835
    }
  }
}
```

```
$ cat doc.ini | magika -
-: INI configuration file (text)
```

```
$ magika --help
Determines the content type of files with deep-learning

Usage: magika [OPTIONS] [PATH]...
```

```
Arguments:
  [PATH]...
    List of paths to the files to analyze.

    Use a dash (-) to read from standard input (can only be used once).
```

```
Options:
  -r, --recursive
    Identifies files within directories instead of identifying the directory itself

  --no-dereference
    Identifies symbolic links as is instead of identifying their content by following them

  --colors
    Prints with colors regardless of terminal support

  --no-colors
    Prints without colors regardless of terminal support

  -s, --output-score
    Prints the prediction score in addition to the content type

  -i, --mime-type
    Prints the MIME type instead of the content type description

  -l, --label
    Prints a simple label instead of the content type description

  --json
    Prints in JSON format

  --jsonl
    Prints in JSONL format

  --format <CUSTOM>
```

Prints using a custom format (use `--help` for details).

The following placeholders are supported:

```
%p The file path
%l The unique label identifying the content type
%d The description of the content type
%g The group of the content type
%m The MIME type of the content type
%e Possible file extensions for the content type
%s The score of the content type for the file
%S The score of the content type for the file in percent
%b The model output if overruled (empty otherwise)
%% A literal %
```

```
-h, --help
    Print help (see a summary with '-h')
```

```
-V, --version
    Print version
```

Python module and other bindings

While the command line client is handy for many use cases, it may not be the most suitable for automated workflows. Thus, Magika comes with bindings for Python and other languages (see the [Bindings section](#) below for more details).

Here is a few examples on how to use the `Magika` Python module:

```
>>> from magika import Magika
>>> m = Magika()
>>> res = m.identify_bytes(b'function log(msg) {console.log(msg);}')
>>> print(res.output.label)
javascript
```



```
>>> from magika import Magika
>>> m = Magika()
>>> res = m.identify_path('./tests_data/basic/ini/doc.ini')
>>> print(res.output.label)
ini
```



```
>>> from magika import Magika
>>> m = Magika()
>>> with open('./tests_data/basic/ini/doc.ini', 'rb') as f:
>>>     res = m.identify_stream(f)
>>> print(res.output.label)
ini
```



Please consult the [python documentation](#) for details on the Python `Magika` API, and the [additional documentation listed below](#) for more information about the output format and other aspects.

Documentation

- [Core concepts: models, prediction mode, magika's output, and content types knowledge base](#)
- [List of supported content types by the latest model, standard_v3_3](#)
- [Documentation about the CLI](#)
- [Documentation about the python `magika` package and module](#)
- [Frequently Asked Questions](#)

Bindings

Artifact	Status	Latest version	Default model
Python <code>Magika</code> module	Stable	0.6.1	standard_v3_2
Rust <code>magika</code> CLI	Stable	0.1.1	standard_v3_2
Rust <code>magika</code> library	Stable	0.1.1	standard_v3_2

Artifact	Status	Latest version	Default model
TypeScript / NPM package (README & docs)	Stable	0.3.2	standard_v3_3
Demo Website	Stable	-	standard_v3_3
GoLang	In development	-	-

Development Setup

We have Magika bindings in multiple languages; each of them has its own development setup. Consult the documentation associated to each binding for more information. For example, for the python `magika` package and module, consult [python/README.md](#).

Known Limitations & Contributing

Magika significantly improves over the state of the art, but there's always room for improvement! More work can be done to increase detection accuracy, support for additional content types, bindings for more languages, etc.

This initial release is not targeting polyglot detection, and we're looking forward to seeing adversarial examples from the community. We would also love to hear from the community about encountered problems, misdetections, features requests, need for support for additional content types, etc.

Check our open GitHub issues to see what is on our roadmap and please report misdetections or feature requests by either opening GitHub issues (preferred) or by emailing us at magika-dev@google.com.

NOTE: Do NOT send reports about files that may contain PII!

See [CONTRIBUTING.md](#) for details.

Additional Resources

- [Google's OSS blog post](#) about Magika announcement.
- Web demo: [web demo](#).

🔗 Research Paper and Citation

We describe how we developed Magika and the choices we made in our research paper, which was accepted at the International Conference on Software Engineering (ICSE) 2025. A pre-print of our paper is available on arxiv: <https://arxiv.org/abs/2409.13768>.

If you use this software for your research, please cite it as:

```
@InProceedings{fratantonio25:magika,  
  author = {Yanick Fratantonio and Luca Invernizzi and Loua Farah and Kurt Thomas and Marina Zhang and Ange Albertin:  
  title = {{Magika: AI-Powered Content-Type Detection}},  
  booktitle = {Proceedings of the International Conference on Software Engineering (ICSE)},  
  month = {April},  
  year = {2025}  
}
```



Security vulnerabilities

Please contact us directly at magika-dev@google.com

License

Apache 2.0; see [LICENSE](#) for details.

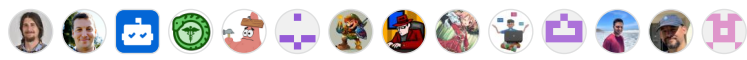
3 weeks ago

[+ 3 releases](#)

Packages

No packages published

Contributors 32



[+ 18 contributors](#)

Deployments 159

✔ **github-pages** 18 hours ago

[+ 158 deployments](#)

Languages

● **Python** 32.1% ● **Rust** 27.2% ● **TypeScript** 17.9% ● **Rich Text Format** 5.8% ● **Vue** 4.2% ● **HTML** 4.1% ● **Other** 8.7%